

Agentic AI Framework for Adaptive Software Development Lifecycle Management

TEAM MEMBERS:-

M. VAISHNAVI - 2420030665

VOKITHA T - 2420030462

P.ADHITYA - 2420030520

T.SHREYAS REDDY - 2420090103

PROJECT DOCUMENTATION:-

Table of Contents

1. Project Title
2. Problem Statement
3. Objectives
4. Abstract
5. Technologies Used
6. Project Modules
7. System Architecture
8. Database Design
9. API Details
10. Testing
11. Future Scope
12. References

2) PROBLEM STATEMENT:-

Software development projects frequently experience changing requirements, unexpected bugs, changing priorities, testing failures, and evolving project conditions. Traditional SDLC processes often require developers and project managers to manually analyze these changes and update requirements, tasks, development plans, and testing activities.

The proposed project uses an **Agentic AI framework** to make the software development lifecycle more adaptive. AI agents can monitor project activities, analyze changes, identify their impact, update development tasks, assist with testing and bug analysis, and provide recommendations. The goal is to reduce manual effort and allow the development process to respond dynamically to changes.

3)OBJECTIVES:-

- Automatically monitor software development activities and project changes.
- Detect changes in requirements and analyze their impact.
- Dynamically adapt development plans and task priorities.
- Use AI agents to assist in coding, testing, debugging, and project management.
- Detect bugs and analyze software quality issues.
- Generate intelligent recommendations for developers.

- Maintain project history and track adaptive changes.
- Improve development speed, flexibility, and consistency.

4) ABSTRACT:-

Agentic AI Framework for Adaptive Software Development Lifecycle Management is an intelligent software engineering system designed to make the Software Development Lifecycle more flexible and responsive to changing project requirements. The system uses multiple AI agents that work together to monitor requirements, planning, development, testing, bugs, risks, and project progress.

When a requirement or project condition changes, the framework analyzes the change, identifies affected components, updates tasks and priorities, and recommends appropriate actions. AI agents can assist with development activities, generate or update test cases, analyze failures, and provide recommendations for resolving issues.

Unlike a traditional fixed SDLC approach, the proposed framework continuously uses feedback from development and testing activities to adapt the development process. This creates a feedback loop of **observe, analyze, decide, act, and adapt**, helping development teams respond more efficiently to changing requirements and software issues.

5)TECHNOLOGIES:-

You can use a stack such as:

- **Frontend:** React.js
- **Backend:** FastAPI / Python
- **AI:** Gemini API
- **Agent Framework:** LangGraph / LangChain

- **Database:** SQLite / PostgreSQL
- **Version Control:** Git & GitHub
- **Automation:** GitHub Actions
- **API:** REST APIs

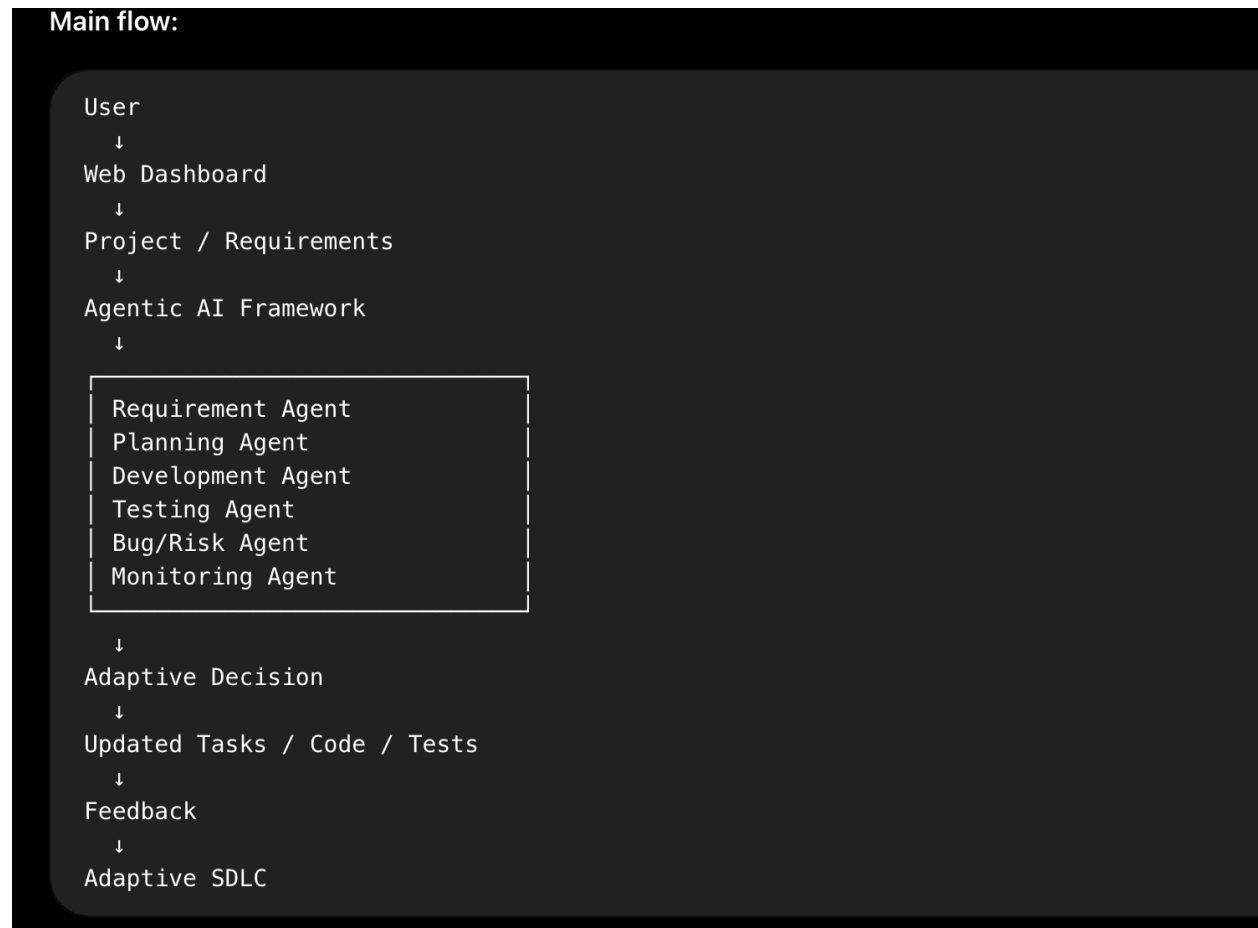
6)PROJECT MODULES:-

- User Authentication
- Project Management
- Requirement Management
- Requirement Change Detection
- AI Agent Management
- Adaptive Planning
- AI-Assisted Development
- Automated Testing
- Bug and Risk Analysis
- Feedback and Adaptation
- Dashboard and Monitoring
- Report Generation

7)SYSTEM ARCHITECTURE:-

The user creates or selects a software project and enters its requirements. The system monitors project activities and sends relevant information to the AI agent framework. Different agents analyze requirements, planning, development, testing, bugs, and risks.

When a change is detected, the agents communicate with each other to determine its impact and adapt the project plan. The updated tasks and recommendations are then displayed on the dashboard. Testing results and developer feedback are fed back into the system so that the development process can adapt again.



8)DATABASE DESIGN:-

The database can store:

- User information
- Project details
- Requirements
- Requirement changes
- Development tasks
- AI agent activities

- Code/repository information
- Test cases and results
- Bugs and risks
- AI recommendations
- Adaptation history
- Project activity history

9)API DETIALS:-

The backend can provide REST APIs for:

- User login and authentication
- Project creation and management
- Requirement creation and modification
- Requirement change detection
- AI agent execution
- Task generation and prioritization
- Code analysis
- Test generation and execution
- Bug analysis
- Report generation
- Project history and adaptation history

10)TESTING:-

The project can undergo:

- Functional testing
- API testing
- Integration testing

- AI agent testing
- User interface testing
- Requirement-change testing
- Adaptive workflow testing
- Basic performance testing

THE WORKFLOW EXAMPLE:-

Add New Requirement



Requirement Agent detects change



Impact Analysis



Planning Agent updates tasks



Testing Agent updates tests



Results analyzed



Development plan adapted

11)FUTURE SCOPE:-

Future improvements may include:

- Autonomous code modification
- Support for multiple AI agent frameworks
- Integration with Jira and other project-management tools
- Advanced GitHub integration
- Complete CI/CD pipeline automation
- Security vulnerability detection
- Predictive software quality analysis
- Team collaboration
- Automatic sprint planning
- Intelligent effort and time estimation
- Self-improving development agents

12)REFERENCES:-

- React.js Documentation
- FastAPI Documentation
- Google Gemini API Documentation
- LangChain / LangGraph Documentation
- GitHub REST API Documentation
- GitHub Actions Documentation
- Software Engineering and Agile/Adaptive Development research papers